The Boundary of Graph Sparsification for Community Detection

Mohammad Dindoost, Bavan DivaaniAazar, Ioannis Koutis, and David A. Bader

Abstract—

Index Terms—Graph sparsification, community detection, modularity, evaluation methodology, graph reduction.

I. Introduction

Community detection is a fundamental task in network analysis [1], [2], and a computationally demanding one. The most widely used methods, including modularity optimization [3], [4], [5] and flow-based approaches [6], have running times that scale with the number of edges, and on the graphs to which they are now routinely applied that quantity is large [7].

Graph sparsification constructs a subgraph on the same vertex set with substantially fewer edges while preserving properties of the original. The strongest guarantees are spectral. Spielman and Srivastava [8] show that sampling each edge with probability proportional to its effective resistance, and reweighting the retained edges by the inverse of that probability, yields a subgraph whose Laplacian quadratic form approximates that of the original within a factor $1 \pm \epsilon$ [9], [10]. Spectral approximation is a statement about the Laplacian, and the eigenvectors of the Laplacian encode cuts and connectivity [11], [12]. Community structure is closely related to those quantities without being identical to them. Because effective resistances are themselves expensive to compute, practical variants approximate them. DSpar [13] replaces the resistance of an edge (u, v) by $1/d_u + 1/d_v$, which bounds it within a factor $2/\alpha$ where α is the normalized spectral gap [14], and thereby obtains a spectral sparsifier at the cost of reading the degree sequence. If the properties preserved by sparsification include community structure, then detection can be performed on the sparsified graph at lower cost and without loss of quality. Whether they do is the question this paper addresses. The answer we obtain is conditional. On the networks and detection algorithms examined here, two conditions have to hold together: the algorithm must take the number of communities as an input, and the graph must carry edges whose removal does not destroy its community structure. Where either condition fails, we find no benefit of any kind, in quality or in speed.

Satuluri et al. [15] introduced local similarity sparsification, in which each vertex retains the incident edges whose endpoints have the highest Jaccard similarity of neighbourhoods. They reported speedups commonly between $10\times$ and $50\times$ with little or no loss of cluster quality, and improved accuracy for two of the four clustering algorithms they tested.

The approach was taken up and extended. Hamann et al. [16] conducted a systematic comparison of structure-preserving sparsification methods for social networks and released reference implementations, which are now distributed in standard graph libraries. Wu and Chen [17] trained a generative model to sparsify graphs specifically for community detection, and report that clustering accuracy is retained while as little as five percent of the edges are kept. Sparsification also entered production: the SimClusters system at Twitter [18] prunes its interaction graph before community detection at industrial scale. Most recently, Chen et al. [19] benchmarked twelve sparsification algorithms against sixteen graph properties on fourteen networks, concluding that no single sparsifier preserves all properties and that the choice should be matched to the downstream task.

The conditions under which the original result was obtained are specific, and none of them travelled with it. The accuracy gain was reported for fixed-k cut partitioners, judged against external ground-truth communities, on dense graphs, and against clustering runs that took hours. The pipeline is now applied to modularity optimizers that choose their own granularity, judged partly by objectives computed on the graph itself, on sparse graphs, and against detection algorithms that run in near-linear time. The same paper reports a synthetic sweep in which the method begins to outperform clustering on the unsparsified graph at an average degree of approximately 50, and to our knowledge no subsequent work examines that threshold.

These extensions also change what has to be measured. The question a sparsify-then-detect pipeline is meant to answer is whether the communities of the original graph can still be recovered after most of its edges are removed. The object of interest is therefore the original graph, and the quantity to be compared is the quality, on that graph, of two partitions: the one obtained by running the detection algorithm on the sparsified graph, and the one obtained by running it on the original graph directly. Both partitions cover the same vertex set, so both can be scored on the same object, and the difference between those two scores is the effect of sparsification.

The sparsifier classes and detection algorithms described above each impose further requirements of the same kind,

The authors are with the Department of Computer Science, New Jersey Institute of Technology, Newark, NJ, USA. E-mail: {md724, bd344, ikoutis, bader}@njit.edu

and several have been noted individually in the literature. Similarity-based sparsifiers [15] remove between-community edges preferentially, and those are the edges that modularity and conductance penalize, so the sparsifier and the quality measure act on the same quantity. Degree-based sparsifiers [13] select on endpoint degrees, and a heavy-tailed degree sequence reproduces several of the resulting effects with no community structure present, so a degree-preserving random graph [20] is needed to separate the two. Detection algorithms that choose their own granularity [4], [5] return finer partitions on a sparsified graph: Chen et al. [19] document community counts rising by three orders of magnitude across a pruning sweep, and Hamann et al. [16] warn that normalized mutual information is inflated when sparsification fragments the graph, recommending chance-corrected indices in its place. Finally, sparsification is itself a computation, and Gottesbüren et al. [21] observe that its overhead can counteract the speedup it is intended to produce. Each requirement is a property of a specific method class rather than a general caveat, and each bounds what a comparison involving that class can conclude. Section III states the requirements and the control that establishes each one.

Applying these requirements, we re-examined the question across seven sparsifiers, spanning the degree-based and similarity-based families that have claimed quality gains, the method of Satuluri et al. among them, and a connectivity-preserving class that cannot fragment a graph by construction; four detection algorithms drawn from the two families identified above; seventeen real networks of up to 25 million edges; and a synthetic benchmark in which average degree is swept over an order of magnitude while community structure is held fixed. One arm of the synthetic study uses a fixed-k partitioner, so that the number of communities is identical in the sparsified and unsparsified conditions and granularity is excluded by construction rather than by matching. The study comprises 22 controlled experiments. Predictions and stopping rules were fixed before each experiment was run, and several of our own hypotheses did not survive them.

Of the two conditions, the second has a location. Average degree is the correlate we can vary under control, and measured against it the transition between the two regimes is sharp within a given family of graphs while its location is not universal. On the LFR benchmark graphs [22] we generate it falls near an average degree of 50, which is also the threshold reported in the original synthetic sweep. On the real networks we test it has already occurred at an average degree of 29, the lowest density at which we tested a fixed-k partitioner. We therefore state a condition rather than a threshold. Density itself is unlikely to be the operative variable. Injecting spurious edges into a graph makes sparsification more effective, which points instead to how many edges can be removed without destroying community structure. Degree heterogeneity and the noise present in how a graph was collected are further correlates of that quantity, and we do not separate them here.

Where either condition fails, no benefit of any kind is present. For detection algorithms that choose their own granularity, on the sparse real networks we test, no sparsifier improves modularity measured on the original graph beyond run-to-run variation, at any retention that preserves quality, and this holds for every such algorithm we tested. There is no speed benefit either. Measured end to end, including the cost of the sparsifier, the pipeline is no faster than clustering the original graph directly, and at aggressive retention it is slower, because a fragmented graph presents the optimizer with more work rather than less. Every apparent gain we found in this regime dissolves when granularity is matched, when chance is subtracted, or when the baseline is given an equal compute budget, with one exception on one network, where the surviving recovery gain is instead exceeded by an unsparsified run of a different detection algorithm.

Where both conditions hold, the gain is real. On synthetic graphs with planted communities, with the number of communities fixed by construction so that no granularity effect is possible, similarity-based sparsification raises both the objective measured on the original graph and chance-corrected agreement with the planted labels. Recovery turns positive at a lower density than the objective does. Degree-based and uniform sparsification at identical retention do not reproduce the effect, so it belongs to the similarity signal rather than to the edge budget.

A fixed-k partitioner applied to a sparse graph shows no gain on the objective in any configuration we tested, and a free-granularity optimizer applied to a dense one shows none either once granularity is controlled. Neither condition is sufficient on its own. Which sparsifier is used matters less than either condition, provided its selection signal reflects local structure rather than degree alone. The mechanism is consistent with this: what governs a degree-based sparsifier's effect on modularity is where high-degree-product edges sit relative to community boundaries, which we establish by direct intervention.

Two qualifications attach to this result. Sparsification repairs a weaker detector rather than surpassing a stronger one. On the synthetic benchmarks, a resolution-tuned modularity optimizer on the untouched graph outperforms every sparsified pipeline we measured, and the same holds on all but one of the real networks we test. And with an exact similarity computation the quality gain is not a speed gain, because the sparsifier costs more than the detection it accelerates.

This paper contributes the following.

- The boundary. The conditions under which sparsification helps, namely a detection algorithm that takes the number of communities as an input and a graph carrying sufficient removable redundancy, established under controls that exclude granularity by construction, together with the negative territory delimited across seven sparsifiers and four detection algorithms on seven real networks. The negative result is not an artifact of the fragmentation that sparsification induces: a connectivity-preserving sparsifier that cannot disconnect a graph leaves no vertex in a small component, and modularity falls by the same margin. We also report that these methods have a hard retention floor on sparse graphs, so that aggressive sparsification is not merely unhelpful there but unreachable.

- An evaluation protocol, and the measured cost of omitting it. Each control is motivated by a specific failure it catches, and the consequence of the central omission is quantified at 45 sign reversals in 63 configurations. We also introduce a permuted-weight control for similarity-weighted pipelines, which shows that the benefit attributed to similarity weighting is reproduced, and in one case exceeded, when the weights are randomly permuted across edges.
- A mechanism for the boundary. A decomposition of the score-separation quantity into a sorting term and a granularity term across seventeen networks, and a direct causal test in which that quantity is steered through more than a full sign change while degree sequence, partition, intra-edge fraction and modularity are all held exactly fixed.
- An open phenomenon. In several controlled settings the partitions that best recover reference communities are not the partitions that score best on the objective, and the two criteria move in opposite directions within the same configuration. We report this as an observation rather than a result, and argue that it is the most consequential open question the boundary exposes.

II. Background and Setting

III. Evaluation Protocol

Section I listed four requirements, each belonging to a specific class of method rather than to sparsification in general. This section states them as controls. Each is given with the failure it catches and with what we measure when it is absent. Table III collects them. Every number in the sections that follow is produced under all of them.

A. One yardstick for two partitions

Detection may use any graph; scoring must use the original one. Let $\mathcal{A}$ be a detection algorithm, G the original graph and G' a sparsified copy of it. The two partitions under comparison are $P_{\text{sparse}} = \mathcal{A}(G')$ and $P_{\text{base}} = \mathcal{A}(G)$, and since both are partitions of the same vertex set, the question the pipeline raises is which of them better describes G. Two accountings of that question appear in this literature:

$$\Delta Q_{\text{transfer}} = Q(P_{\text{sparse}}, G) - Q(P_{\text{base}}, G), \tag{1}$$
$$\Delta Q_{\text{self}} = Q(P_{\text{sparse}}, G') - Q(P_{\text{base}}, G). \tag{2}$$

They differ in one place, the graph on which the first term is evaluated. Equation (1) measures both partitions with a single instrument. Equation (2) changes the instrument between the arm under test and the arm it is tested against, and the instrument it applies to the former is the graph from which edges have been removed. We report $\Delta Q_{\text{transfer}}$ throughout; ΔQ_{self} appears in released benchmark tooling and in recent work in this line, documented in Section VI.

TABLE I
Uniform random edge deletion under the two accountings, Leiden, mean of three seeds. ΔQ_{self} is positive in every cell and $\Delta Q_{\text{transfer}}$ is negative in every cell. Uniform deletion uses no structural information, so the inflation comes from the objective rather than from any selection rule.

Network	$Q(P_{\text{base}}, G)$	ρ	ΔQ_{self}	$\Delta Q_{\text{transfer}}$
ca-CondMat	0.729	0.800	+0.0111	−0.0164
ca-CondMat	0.729	0.900	+0.0053	−0.0064
email-Enron	0.611	0.800	+0.0052	−0.0162
email-Enron	0.611	0.900	+0.0011	−0.0086
com-DBLP	0.826	0.800	+0.0078	−0.0247
com-DBLP	0.826	0.900	+0.0040	−0.0098

The two accountings diverge systematically, and the objective shows why. Modularity scores a partition P of a graph with m edges by

$$Q(P, G) \;=\; \sum_{C \in P} \left[\frac{m_C}{m} - \gamma \left(\frac{D_C}{2m} \right)^2 \right], \tag{3}$$

with m_C the number of edges inside community C, D_C the sum of degrees in C, and γ a resolution parameter equal to one in the standard definition. The subtracted term is normalized by m. Deleting edges shrinks it, so modularity computed on a sparsified graph rises whether or not the partition has improved, and the rise is larger the more edges are removed.

The effect is not small and it is not confined to sparsifiers that select intelligently. Table I scores uniform random edge deletion, whose selection rule uses no information at all, both ways on three networks at two retention levels. The sign of the conclusion is opposite in every one of the six cells, and the gap between the two accountings roughly doubles when retention falls from 0.9 to 0.8, which is the behaviour (3) predicts.

Across a larger grid the pattern holds under every algorithm we tested. Table II covers 63 configurations spanning three detection algorithms, seven networks and three sparsifier settings. Modularity read off the sparsified graph is positive in 60 of them and reaches +0.537; the same partitions scored on the original graph are positive in 15. The sign of the conclusion differs between the two accountings in 45 of the 63 configurations. Eleven cells exceed a compute-matched baseline, all of them under label propagation, and they are not a subset of the 15: two of them sit below the plain baseline mean while beating a matched control that had drawn too few restarts, which is the failure Section III-D treats.

B. Matching granularity

Sparsification fragments a graph, and an algorithm that chooses its own granularity responds by returning more

TABLE II
The two accountings across 63 configurations, seven networks by three sparsifier settings under each algorithm. A flip is a cell where $\Delta Q_{\text{self}} > 0$ and $\Delta Q_{\text{transfer}} < 0$, so that the two accountings disagree on the sign of the conclusion.

Algorithm	Cells	$\Delta Q_{\text{self}} > 0$	$\Delta Q_{\text{transfer}} > 0$	Flips	max ΔQ_{self}
Infomap	21	20	6	14	+0.312
Louvain	21	20	0	20	+0.260
Label propagation	21	20	9	11	+0.537
Total	63	60	15	45	+0.537

communities. Chen et al. [19] report community counts rising by three orders of magnitude across a pruning sweep. Best-match F_1, purity and normalized mutual information all increase with the number of communities, so a recovery comparison between partitions of different granularity measures that difference along with whatever else it is meant to measure. Hamann et al. [16] published the warning for normalized mutual information and recommended chance-corrected indices in its place; the control below operationalizes it.

The control is a partition of the original graph at a matched community count, obtained by raising the resolution parameter γ in (3) until the counts agree. On com-DBLP, Louvain returns 220 communities on the original graph and scores 0.102 average best-match F_1 against the reference communities. Local similarity sparsification at retention 0.5 returns 10,947 communities and scores 0.193, an apparent gain of +0.091. The original graph partitioned at $\gamma = 576$ returns 11,159 communities and scores 0.300. The finer partition is worth twice what the sparsification appeared to be worth, and it requires no sparsifier. Leiden reproduces both numbers to three decimal places, which places the effect in the modularity objective's default granularity rather than in one optimizer's search.

Where the counts cannot be brought together, we draw no recovery conclusion. The comparability rule is $|\Delta k|/k < 25\%$, and the cells it excludes are reported as excluded rather than compared.

C. Subtracting chance

Recovery is reported with chance-corrected indices. Where an uncorrected measure is used because the reference communities are overlapping and numerous, as with average best-match F_1 on the largest networks, we also compute the score of a random partition with the same community size distribution, and report it beside the measurement.

That floor moves with the quantity the previous control governs. On com-Amazon under Louvain, a size-matched random partition scores 0.009 against the reference communities at 231 communities and 0.057 at 8,651, a 6.6× rise produced by fragmentation alone. Any comparison of best-match F_1 across different community counts reads part of that slope, and subtracting the floor is what separates the two.

D. An equal compute budget

Every detection algorithm we use is stochastic, so a single run is a draw rather than an answer, and a pipeline that sparsifies and then detects has spent more time than one detection run. The registered control gives the baseline as many independent restarts as fit the pipeline's wall clock and takes the best of them.

That control is weaker than it appears, and its weakness favours the treatment. When the pipeline is fast the budget buys very little: two restarts in all 48 cells of one sweep, and between 1 and 16 in another, with 1 in most cells on the two largest networks. Label propagation shows what this costs. Under the compute-matched control it is the strongest of the three algorithms in that sweep, with a mean change of +0.009; against the best of 20 plain restarts of the same baseline it is the weakest, at −0.106. On email-Enron its two apparent gains of +0.146 and +0.161 become −0.068 and −0.053, because the compute-matched baseline had drawn 1 to 2 restarts and never sampled the algorithm's good mode, whose modularity is 0.523 against the matched control's 0.308.

We therefore report both: the compute-matched figure, which is the fair accounting of what the pipeline costs, and the best of a fixed number of restarts, which is a strictly more generous baseline that can only handicap the treatment. A result counts as surviving only if it survives both. This is also the reason the granularity control above is applied to results that have already passed the compute test, rather than in place of it.

E. Nulls

Two of the quantities this literature reads as evidence of community structure can be produced without any community structure at all. Each has a null that detects the substitution.

a) A degree sequence without communities: A degree-based sparsifier selects on a function of the degree sequence, so an effect it produces on a heavy-tailed graph may follow from that sequence alone. The control is a degree-preserving rewiring [20], built by double edge swaps, which holds every vertex degree exactly and destroys the community structure. A signal that reappears on the rewiring is not evidence about communities.

Two quantities fail this test. The first is the change in modularity of a fixed partition when the graph is sparsified, which is positive on all seventeen networks we tested; on their rewirings it is positive on all seventeen as well, and larger than the real value on fifteen of them. The second is the separation between the scores a degree-based sparsifier assigns to within-community and between-community edges, which on the rewirings is larger than on the real graphs in sixteen of seventeen cases, with a median ratio of 1.60. Real community structure suppresses that separation rather than producing it. Neither quantity is used as evidence in this paper. The mechanism they belong to is the subject of Section V, where the same null is what makes the causal test interpretable.

b) Weights without their placement: A related family of proposals keeps every edge and reweights it by a similarity score. The corresponding null permutes the computed weights uniformly at random across the edges, which preserves the weight distribution exactly and destroys every association between a weight and the structure that produced it. We are not aware of its use in this literature.

It changes the reading of the one modularity gain we obtained from four ways of deploying a similarity signal. On email-Enron, weighting each edge by $1 + J(u, v)$, with J the Jaccard similarity of the endpoint neighbourhoods, raises modularity measured on the original graph to 0.6153 ± 0.0034 against a baseline of 0.6071 and a compute-matched best of 0.6051, a gain of $+0.0102$ at five pooled standard deviations. Permuting those same weights across edges gives 0.6168 ± 0.0020, a gain of $+0.0117$, which is larger. On a degree-preserving rewiring of ca-CondMat, where the similarity signal is absent by construction, weighting still produces $+0.0023$ and clears the same significance test. What the weights contribute is heterogeneity in the optimizer's search rather than information about communities.

F. Realized retention

Retention is measured, never assumed. The sampling scheme in common use clips per-edge probabilities at one, and between 13% and 35% of edges reach that clip, so the expected number of retained edges falls below the nominal αm and saturates: at $\alpha = 1.0$ one of our networks retains 0.662 of its edges, and at $\alpha = 0.95$ another retains 0.447. Three samplers that all describe themselves by the same α realize 0.33 to 0.55, 0.37 to 0.68, and, once the scale factor is solved by bisection, 0.70 to 0.95. Sparsifiers are compared at matched realized retention throughout, and the realized value appears in every table.

Two properties of the local selection rules limit what can be matched. The achievable retentions form a coarse grid, so that on com-Amazon local similarity sparsification can produce only 0.301 and 0.542 and nothing between them. And several methods cannot reach aggressive retention on a sparse graph at all, because each vertex keeps at least one incident edge; the resulting floors reach 0.36 on the sparsest network we test, which is reported with the results rather than treated as a free parameter.

A comparison must also be like for like in what is removed. A method that deletes vertices as well as edges scores its partition on a smaller and denser subpopulation, so the fraction of vertices retained belongs in the table beside the fraction of edges. Every sparsifier in this study removes edges only, and we report both fractions.

G. Cost measured end to end

The sparsifier is a computation and is charged to the pipeline that uses it, as it was in the original report [15] and as Gottesbüren et al. [21] recommend after measuring the same overhead in graph partitioning. The comparison is the whole pipeline, sparsifier included, against one run of the same detection algorithm on the original graph.

Among the cells of our largest sweep that preserve quality, the greatest end-to-end speedup is $0.66\times$, which is slower than not sparsifying. Speedups above $1.5\times$ occur in 12 of 63 cells, and every one of them loses modularity. Discounting the sparsifier entirely and timing only the detection step, the median speedup is between $1.15\times$ and $1.63\times$ by algorithm, well short of the factor of two that halving the edges suggests, because all three algorithms return more communities on a sparsified graph and do more work per pass.

H. Pre-registration

Each experiment in this study has a design document written before its code, fixing what is measured, what is predicted, and what outcome would end the line of work. Several of the predictions were ours and failed. The synthetic sweep that locates the boundary was registered with the prediction that sparsification would not produce an honest modularity gain at any density, and the fixed-k arm falsified it.

IV. Below the Boundary: Where Sparsification Does Not Help

V. The Mechanism

VI. Related Work

VII. Discussion

VIII. Conclusion

References

[1] S. Fortunato, "Community detection in graphs," Physics Reports, vol. 486, no. 3–5, pp. 75–174, 2010.

[2] S. Fortunato and M. E. Newman, "20 years of network community detection," Nature Physics, vol. 18, no. 8, pp. 848–850, 2022.

[3] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," Physical Review E, vol. 69, no. 2, p. 026113, 2004.

[4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, no. 10, p. P10008, 2008.

[5] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," Scientific Reports, vol. 9, no. 1, pp. 1–12, 2019.

[6] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," Proceedings of the national academy of sciences, vol. 105, no. 4, pp. 1118–1123, 2008.

[7] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, "Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters," Internet Mathematics, vol. 6, no. 1, pp. 29–123, 2009.

[8] D. A. Spielman and N. Srivastava, "Graph sparsification by effective resistances," in Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing, 2008, pp. 563–568.

[9] A. A. Benczúr and D. R. Karger, "Randomized approximation schemes for cuts and flows in capacitated graphs," SIAM Journal on Computing, vol. 44, no. 2, pp. 290–319, 2015.

[10] D. A. Spielman and S.-H. Teng, "Nearly linear time algorithms for preconditioning and solving symmetric, diagonally dominant linear systems," SIAM Journal on Matrix Analysis and Applications, vol. 35, no. 3, pp. 835–885, 2014.

TABLE III
The protocol. Each control is motivated by a specific failure, and the last column reports what we measure when the control is absent.

Failure	Control	Measured consequence of omitting it
Quality read off the sparsified graph	Score both partitions on the original graph, Eq. (1)	The sign of the conclusion differs in 45 of 63 configurations; positive in 60 of 63 one way and in 15 of 63 the other (§III-A)
Finer partitions score higher	Partition the original graph at a matched community count	com-DBLP: an apparent +0.091 in best-match F_1 against a control that reaches +0.198 with no sparsifier (§III-B)
Uncorrected agreement measures	Chance-corrected indices, and a size-matched random-partition floor	The floor rises 6.6× between 231 and 8,651 communities on one graph (§III-C)
Unequal compute	Compute-matched restarts and the best of a fixed number	One algorithm ranks first under the former and last under the latter; two gains near +0.15 become losses (§III-D)
Degree mechanics read as structure	Degree-preserving rewiring	Both quantities are reproduced by the null, and are larger on it in 15 of 17 and 16 of 17 networks (§III-E)
Weighting credited to similarity	Permute the weights across edges	The permuted weights reproduce the gain and exceed it, +0.0117 against +0.0102 (§III-E)
Nominal retention	Report and match on measured retention	The same nominal α realizes retentions from 0.33 to 0.95 depending on the sampler (§III-F)
Sparsification charged to nobody	Time the pipeline, sparsifier included	No quality-preserving configuration exceeds 0.66× (§III-G)

[11] U. von Luxburg, “A tutorial on spectral clustering,” Statistics and Computing, vol. 17, no. 4, pp. 395–416, 2007.

[12] J. Shi and J. Malik, “Normalized cuts and image segmentation,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 8, pp. 888–905, 2000.

[13] Z. Liu, K. Zhou, Z. Jiang, L. Li, R. Chen, S.-H. Choi, and X. Hu, “Dspar: An embarrassingly simple strategy for efficient GNN training and inference via degree-based sparsification,” Transactions on Machine Learning Research, 2023.

[14] L. Lovász, “Random walks on graphs,” Combinatorics, Paul Erdős Is Eighty, vol. 2, no. 1–46, p. 4, 1993.

[15] V. Satuluri, S. Parthasarathy, and Y. Ruan, “Local graph sparsification for scalable clustering,” in Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, 2011, pp. 721–732.

[16] M. Hamann, G. Lindner, H. Meyerhenke, C. L. Staudt, and D. Wagner, “Structure-preserving sparsification methods for social networks,” Social Network Analysis and Mining, vol. 6, no. 1, p. 22, 2016.

[17] H.-Y. Wu and Y.-L. Chen, “Graph sparsification with generative adversarial network,” in 2020 IEEE International Conference on Data Mining (ICDM), 2020.

[18] V. Satuluri, Y. Wu, X. Zheng, Y. Qian, B. Wichers, Q. Dai, G. M. Tang, J. Jiang, and J. Lin, “SimClusters: Community-based representations for heterogeneous recommendations at Twitter,” in Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), 2020, pp. 3183–3193.

[19] Y. Chen, H. Ye, S. Vedula, A. Bronstein, R. Dreslinski, T. Mudge, and N. Talati, “Demystifying graph sparsification algorithms in graph properties preservation,” Proceedings of the VLDB Endowment, vol. 17, no. 3, pp. 427–440, 2024.

[20] M. Molloy and B. Reed, “A critical point for random graphs with a given degree sequence,” Random Structures & Algorithms, vol. 6, no. 2–3, pp. 161–180, 1995.

[21] L. Gottesbüren, N. Maas, D. Rosch, P. Sanders, and D. Seemaier, “Linear-time multilevel graph partitioning via edge sparsification,” in 33rd Annual European Symposium on Algorithms (ESA), 2025, arXiv:2504.17615.

[22] A. Lancichinetti, S. Fortunato, and F. Radicchi, “Benchmark graphs for testing community detection algorithms,” Physical Review E, vol. 78, no. 4, p. 046110, 2008.